

OOPs in Python

- ❑ **OOPs in Python** is a programming approach that focuses on using objects and classes as same as other general programming languages.
- ❑ Python allows developers to develop applications using the OOPs approach with the major focus on code reusability.
- ❑ It is very easy to create classes and objects in Python.

List of OOPs concepts

1. Class
2. Object
3. Method
4. Inheritance
5. Polymorphism
6. Encapsulation

Defining a Class in Python

What is a Class?

A **class** is a blueprint (template) used to create objects.
It defines **variables (data)** and **functions (methods)**.

- **Attributes** → Data/Properties
- **Methods** → Functions/Behaviour

- ❑ To create a class, use the “**class**” keyword followed by the class name.
- ❑ The end of the Class name must declare a **colon**.

Basic Syntax

```
class ClassName:  
    def method(self):  
        # code
```

```
# Creating object  
obj = ClassName()
```

```
class Student:  
    def display(self):  
        print("Welcome Student")
```

```
# object creation  
s1 = Student()
```

```
# method call  
s1.display()
```

What is **self**?

self refers to the **current object**
Used inside class methods to access variables

```
2 class myClass():
3     def method1(self):
4         print "Guru! )"
5
6     def method2(self, someString):
7         print "Software Testing:" + someString
8
```

- ❖ **NOTE:** About using "**self**" in Python
- ❖ The self-argument refers to the object itself. Hence the use of the word self. So inside this method, self will refer to the specific instance of this object that's being operated on.
- ❖ Self is the name preferred by convention by Python to indicate the **first parameter** of instance methods in Python. **It is part of the Python syntax to access members of objects**

```
class MyClass:
    a = 10
    b = 20
    def add(self):
        sum = self.a + self.b
        print(sum)
```

What is `self`?

`self` refers to the **current object**

Used inside class methods to access variables

#Creating an object of class MyClass
`ob = MyClass()`

#Accessing function and variables present inside MyClass using the object
`print(ob.a)`
`print(ob.b)`
`ob.add()`

```
RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37/
```

```
py
10
20
30
```

```
class Student:  
    # Method  
    def display_details(self):  
        print("Name:", self.name)  
        print("Age:", self.age)  
        print("Marks:", self.marks)
```

```
# Creating Object  
student1 = Student()
```

```
# Assigning values  
student1.name = "Jaya"  
student1.age = 20  
student1.marks = 85.5
```

```
# Calling method  
student1.display_details()
```

Output

```
Name: Jaya  
Age: 20  
Marks: 85.5
```

Program:3

```
class MyClass:  
    variable = "blah"  
  
    def function(self):  
        print("This is a message inside the class.")
```

```
myobjectx = MyClass() #object1 creation  
myobjecty = MyClass() #object2 creation
```

```
myobjecty.variable = "yackity"
```

```
# Then print out both values  
print(myobjectx.variable)  
print(myobjecty.variable)
```

```
myobjectx.function() #object1 method calling  
myobjecty.function() #object2 method calling
```

RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37,

```
blah  
yackity  
This is a message inside the class.  
This is a message inside the class.
```

Python - Constructor

- ❑ A constructor is a special kind of method which is used for **initializing** the instance variables during **object creation**.
- ❑ constructor always has a name `__init__` and the name `__init__` is **prefixed** and **suffixed** with a double underscore(`__`).
- ❑ declare a constructor using **def** keyword, just like methods.

Syntax of constructor declaration

```
def __init__(self):  
    # body of the constructor
```

Types of constructors in Python

We have two types of constructors in Python.

1. **default constructor** - This constructor doesn't accept any arguments.
2. **parameterized constructor** - constructor with parameters is known as parameterized constructor.

The __init__() Function

All classes have a function called __init__(), which is always executed when the class is being initiated.

Use the __init__() function to assign values to object properties, or other operations

Note: The __init__() function is called automatically every time the class is being used to create a new object.

```
#Declare default constructor
class DemoClass:
    def __init__(self):
        print("Calling Default Constructor")

x=DemoClass()
```

RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37,

Calling Default Constructor

```
#Declare default constructor
class DemoClass:
    a=20
    def __init__(self):
        print("Calling Default Constructor")
        print(self.a)

x=DemoClass()
```

RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37,

Calling Default Constructor
20

Python - Parameterized constructor

- accepts the arguments during object creation then such type of constructors are known as Parameterized constructors.

```
class DemoClass:
    num = 101

    # parameterized constructor
    def __init__(self, data):
        self.num = data

    # a method
    def read_number(self):
        print(self.num)

# creating object of the class
# this will invoke parameterized constructor
obj = DemoClass(55)

# calling the instance method using the object obj
obj.read_number()

# creating another object of the class
obj2 = DemoClass(66)

# calling the instance method using the object obj
obj2.read_number()
```

RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37,

55

66

```
class Employee:
    def __init__(self, name, age, salary, gender):
        self.name = name
        self.age = age
        self.salary = salary
        self.gender = gender

    def employee_details(self):
        print("Name of employee is :", self.name)
        print("Age of employee is :", self.age)
        print("Salary of employee is :", self.salary)
        print("Gender of employee is :", self.gender)

emp = Employee("Raja", 45, 20000, "Male")
emp.employee_details()
```

```
>>>
===== RESTART: C:/Users/JAYAPRAKASH/AppData/Local/Programs/Python/Python38/oopseg1.py
Name of employee is : Raja
Age of employee is : 45
Salary of employee is : 20000
Gender of employee is : Male
>>>
```

Problem Statement: Student Details Management System

Objective:

Write a Python program using **Class and Object** to create and display student details.

Parameterized Constructor

Requirements

Create a class named **Student** with the following attributes:

name

age

marks

The class should contain:

- ❖ A `__init__()` constructor to initialize student details.
- ❖ A method `display_details()` to display the student information.
- ❖ Create **two student objects** and display their details.

Sample Input

Student 1:
Name: Jaya
Age: 20
Marks: 85.5

Student 2:
Name: Kavın
Age: 21
Marks: 90.0

Sample Output

Student Details

Name: Jaya
Age: 20
Marks: 85.5

Student Details

Name: Kavın
Age: 21
Marks: 90.0

Term	Meaning
<code>class</code>	Creates a class
<code>Student</code>	Class name
<code>__init__()</code>	Constructor
<code>self</code>	Refers to the current object
<code>name, age, marks</code>	Attributes
<code>display_details()</code>	Method
<code>student1</code>	Object
<code>student2</code>	Object

```
class Student:
    def __init__(self, name, age, marks):
        self.name = name
        self.age = age
        self.marks = marks

    def display_details(self):
        print("Name:", self.name)
        print("Age:", self.age)
        print("Marks:", self.marks)
```

Creating Objects

```
student1 = Student("Jaya", 20, 85.5)
student2 = Student("Kavin", 21, 90.0)
```

```
student1.display_details()
print()
```

```
student2.display_details()
```

← → ↻ jupyter.org/try-jupyter/lab/

🗄️ | 📧 Mail - Jayaprakash S... 🌐 Dashboard | OmniL... 🌐 Dashboard | OmniL... 📖 Library Managemen... 🌐 EmarquerCloud - Lo... 📄 How to install and c...

💡 File Edit View Run Kernel Tabs Settings Help

📁 / notebooks / 16.09.2026 /

Name	Modified
• 📄 Untitled.ipynb	22h ago

Intro.ipynb × Untitled.ipynb +

📄 + ✂️ 📄 ▶️ ■ ↻ ⏩ Code ▾

```
[1]: class Student:
    def __init__(self, name, age, marks):
        self.name = name
        self.age = age
        self.marks = marks

    def display_details(self):
        print("Name:", self.name)
        print("Age:", self.age)
        print("Marks:", self.marks)

# Creating Objects
student1 = Student("Jaya", 20, 85.5)
student2 = Student("Kavin", 21, 90.0)

student1.display_details()
print()

student2.display_details()
```

Name: Jaya
Age: 20
Marks: 85.5

Name: Kavin
Age: 21
Marks: 90.0

Problem Statement: Calculator Using Parameterized Constructor

Write a Python program to create a **Calculator** class using a **parameterized constructor**.

The program should:

1. Create a class named `Calculations`.
2. Define a parameterized constructor `__init__(self, n1, n2)` to initialize two numbers.
3. Create an `addition()` method to calculate the sum of the two numbers.
4. Create a `subtraction()` method to calculate the difference between the two numbers.
5. Create an object by passing two numbers to the constructor.
6. Call both methods and display the results.

Sample Input

Enter first number: 200
Enter second number: 100

Sample Output

Object Instantiated
Addition Working
Subtraction Working

Sum: 300
Difference: 100

Concepts Covered

Class → Object → Parameterized Constructor → `__init__()` → `self` → Instance Variables → Methods → Return Value

#Constructor with parameters

class Calculations:

```
def __init__(self,n1,n2):  
    self.num1 = n1  
    self.num2 = n2  
    self.sum=0  
    self.diff = 0  
    print('Object Instantiated')  
  
def addition(self):  
    print('Addition Working')  
    self.sum = self.num1 + self.num2  
    return self.sum  
  
def subtraction(self):  
    print('Subtraction Working')  
    self.diff = self.num1 - self.num2  
    return self.diff
```

```
calcobj = Calculations(200,100)  
calcobj.addition()  
calcobj.subtraction()
```

```
print('Sum : ',calcobj.sum)  
print('Difference : ',calcobj.diff)
```

Object Instantiated
Addition Working
Subtraction Working
Sum : 300
Difference : 100
Enter a value :

Python Destructors - Destroying the Object

- Constructor is used to create and initialize an object, a **destructor** is used to destroy the object and perform the final clean up.
- Although in python we do have **garbage collector** to clean up the memory, but its not just memory which has to be freed when an object is the buffer or cache etc. Hence when we say the **final clean up**, it doesn't only mean cleaning up the memory resources.

Using the `__del__` method

Below we have a simple code for a class Example, where we have used the **`__init__` method to initialize our object**, while we have defined the **`__del__` method to act as a destructor**.

```
class Example:
    def __init__(self):
        print ("Object created");

    # destructor
    def __del__(self):
        print ("Object destroyed");
```

```
# creating an object
myObj = Example();
# to delete the object explicitly
del myObj;
```

```
>>>
RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37/constructor.py
Object created
Object destroyed
```